



第四章 指令系统

4.1 指令系统的发展与性能要求

4.2 指令格式

4.3 操作数类型

4.4 指令和数据的寻址方式

4.5 典型指令

4.6 ARM汇编语言

4.0 指令系统概述



25482+426858=????????????



```
a=25482;  
b=426858;  
c=a+b;
```



```
STORE a 25482;  
STORE b 25482;  
ADD c,a,b;
```



```
010100101001  
011100101010  
100110000010  
....
```

自然语言



高级程序语言



汇编语言



机器语言

4.0 指令系统概述



机器“语言”
二进制：0和1
硬件：逻辑电路

010100101001
011100101010
100110000010
....



“指令”

计算机实现**某个基本操作**的命令

指 令：0101110010101010
基本操作：load r1 1382



“指令系统”
一台计算机的
所有指令
集合

4.0 指令系统概述



软件层抽象

Applications

Compiler

OS

计算机硬件和软件的接口及界面

指令系统

硬件层抽象

Instruction Processing

Input/Output

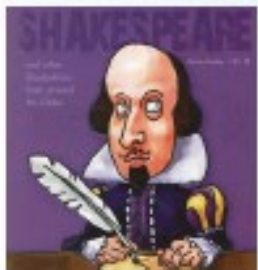
Datapath & Control

Digital Design

Circuit Design



4.0 指令系统概述



There are a thousand Hamlets in a thousand people's eyes.

但是，指令系统的设计大体上**只有两种**，
不是一千种



复杂指令集计算机
Complex Instruction Set Computer



精简指令集计算机
Reduce Instruction Set Computer

4.1 指令系统的发展与性能要求

一、指令系统的发展

1. 指令： 计算机执行某种操作的命令。

机器语言指令： 二进制编码指令

汇编语言指令： 助记符表示的指令

2. 指令系统： 一台计算机中所有机器指令的集合称为这台计算机的指令系统。

4.1 指令系统的发展与性能要求

一、指令系统的发展

3. 系列计算机：基本指令系统相同、基本体系结构相同的一系列计算机。

4. CISC：复杂指令集计算机。

5. RISC：精简指令集计算机。

4.1 指令系统的发展与性能要求

二、对指令系统性能的要求

设计原则

完备性	该有的都要有
有效性	简洁、加速常用操作、没有歧义
规整性	对称、匀齐、一致（简单源于规整）
兼容性	之前/之后的都要能用

4.1 指令系统的发展与性能要求

二、对指令系统性能的要求

一个较完善的指令系统应该包括

数据传送指令	Load/Save 指令
输入输出指令	In/Out 指令
算术运算指令	Add 等指令
逻辑运算指令	And 等指令
系统控制指令	中断等指令
程序控制指令	Jump 等指令



基本指令系统的操作

指令类型	指令助记符	指令操作
数据 传 送	MOV	传送
	STO	存数
	LAD	取数
	EXC	交换
	CLA	清零
	SET	置1
	PUS	进栈
	POP	出栈

指令类型	指令助记符	指令操作
算 术 运 算	ADD	加法
	SUB	减法
	MUL	乘法
	DIV	除法
	ABS	绝对值
	NEG	求负
	INC	加1
	DEC	减1

基本指令系统的操作

指令类型	指令助记符	指令操作
逻辑运算	AND	与
	OR	或
	NOT	反
	EOR	异或
	TES	测试
	COM	比较
	SHI	移位
	ROT	循环移位

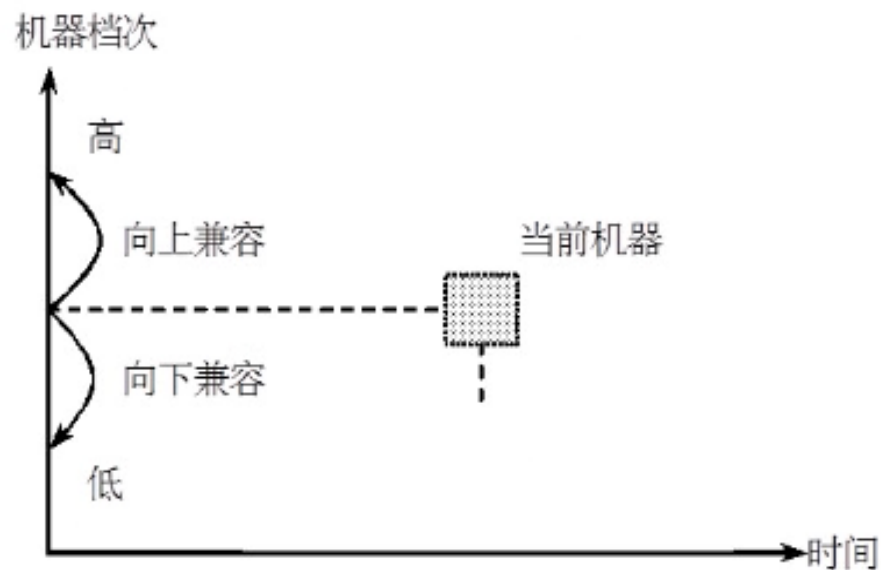
指令类型	指令助记符	指令操作
控制传递	JMP	无条件转移
	JMPX	条件转移
	JMPC	转子程序
	RET	返回

4.1 指令系统的发展与性能要求

二、对指令系统性能的要求



• **向上（下）兼容**：按某档机器编制的程序，不加修改的就能运行于比它高（低）档的机器



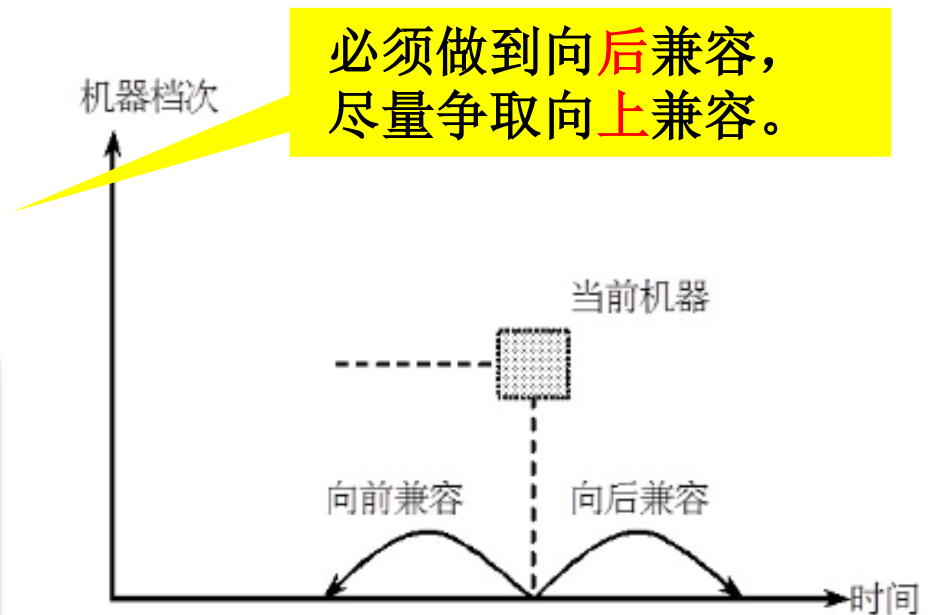
4.1 指令系统的发展与性能要求

二、对指令系统性能的要求

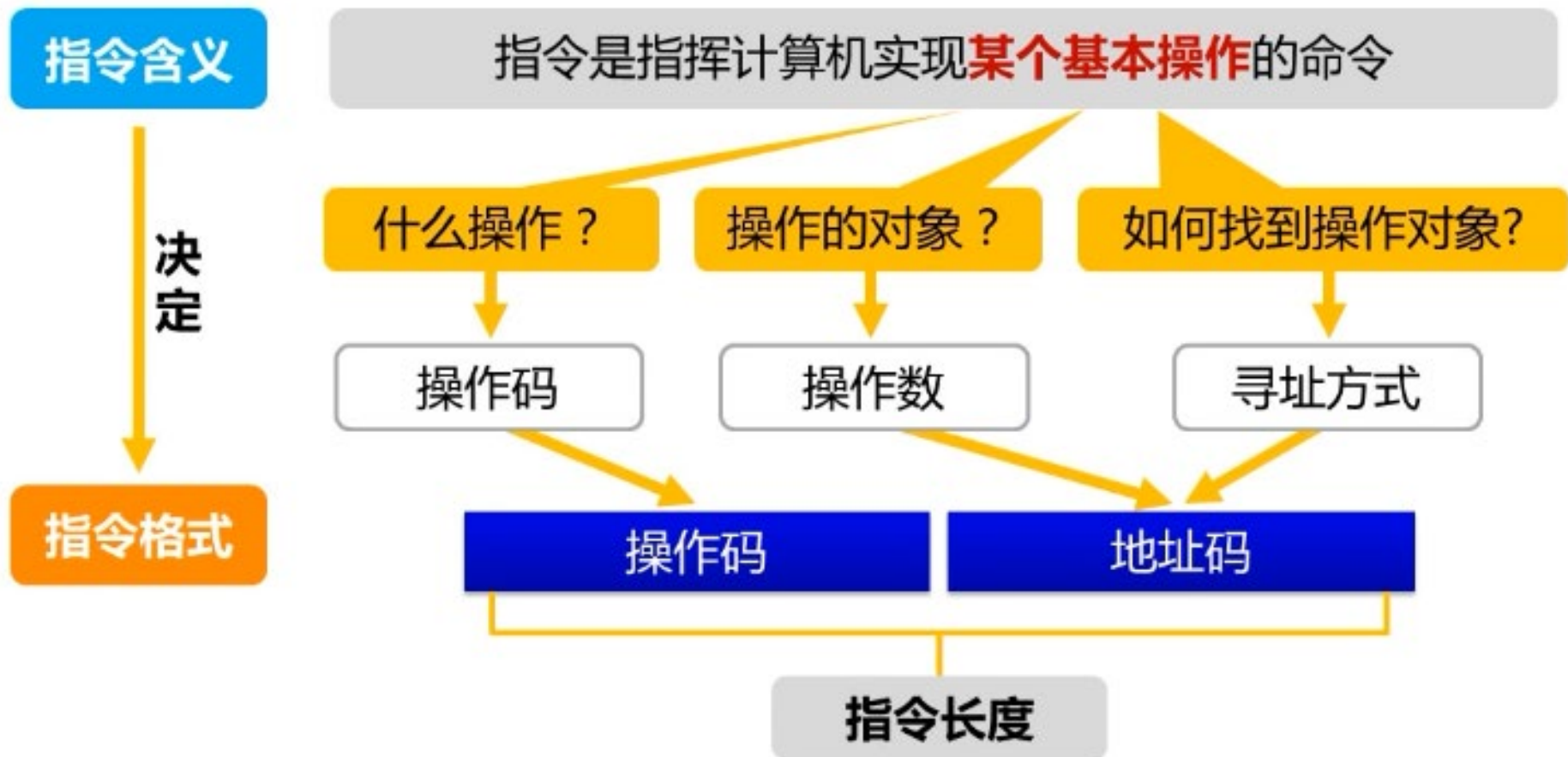


• **向上（下）兼容**：按某档机器编制的程序，不加修改的就能运行于比它高（低）档的机器

• **向前（后）兼容**：按某个时期投入市场的某种型号机器编制的程序，不加修改就能运行于在它之前(后)投入市场的机器



4.2 指令格式



4.2 指令格式

指令格式

操作码

地址码

指令长度的设计

1. 问：每条指令的长度可以是不一样的么？

指令长度

- 一条指令包含的二进制代码位数。
- 取决于操作码长度、操作数地址长度和地址个数。
 - 定长指令字：所有指令的长度相同。需向最长指令看齐。
 - 变长指令字：不同指令的长度不同。

设计原则之一

规整性

4.2 指令格式

指令格式

操作码

地址码

操作码设计

1. 问：每条指令的操作码可以是几个？

4.2 指令格式

指令格式

操作码

地址码

操作码设计

设计原则之一

1. 问：每条指令的操作码可以是几个？

1. 答：只能是一个。

有效性

4.2 指令格式

指令格式

操作码

地址码

操作码设计

设计原则之一

1. 问：每条指令的操作码可以是几个？

1. 答：只能是一个。

有效性

2. 问：具体的操作是怎么表示的？

4.2 指令格式

指令格式

操作码

地址码

操作码设计

设计原则之一

1. 问：每条指令的操作码可以是几个？

1. 答：只能是一个。

有效性

2. 问：具体的操作是怎么表示的？

2. 答：用一定长度的不同编码表示不同的操作

4.2 指令格式

指令格式

操作码

地址码

操作码设计

设计原则之一

1. 问：每条指令的操作码可以是几个？

1. 答：只能是一个。

有效性

2. 问：具体的操作是怎么表示的？

2. 答：用一定长度的不同编码表示不同的操作

设计原则之二

完备性

4.2 指令格式

指令格式

操作码

地址码

操作码设计

设计原则之一

1. 问：每条指令的操作码可以是几个？

1. 答：只能是一个。

有效性

2. 问：具体的操作是怎么表示的？

2. 答：用一定长度的不同编码表示不同的操作

完备性

设计原则之二

3. 问：长度是一定的么？怎么编码？

3. 答：不要着急，等等下一节告诉你。

4.2 指令格式

指令格式

操作码

地址码

1. 问：每条指令的地址码可以是几个？

地址码设计

4.2 指令格式

指令格式

操作码

地址码

1. 问：每条指令的地址码可以是几个？

答：0到多个，看操作码的需要了

一条指令包含1个操作码和多个地址码

- 零地址指令

OP

- 一地址指令

OP	A1
----	----
- 二地址指令(最常用)

OP	A1	A2
----	----	----
- 三地址指令(RISC风格)

OP	A1	A2	A3
----	----	----	----
- 多地址指令

堆栈

累加器

通用寄存器

地址码个数与性能和实现难度密切相关

地址码设计

操作码设计

指令格式

操作码

地址码

操作码长度

操作码长度问题

□ 操作码的编码方式决定操作码的长度

➤ 定长操作码法

➤ 变长/扩展操作码法

□ 操作码长度和指令长度什么关系？

➤ 指令长度是否可变与操作码长度是否可变没有绝对联系

➤ 关注程序代码长度时：变长指令字、变长操作码

➤ 关注性能时：定长指令字、定长操作码

操作码设计

定长操作码

基本思想：指令的操作码部分采用**固定长度**的编码。
例如：假设操作码固定为6位，则系统最多可表示 2^6 种指令。



IBM370指令格式

特点

译码简单，但有信息冗余。

例：IBM360/370采用：8位定长操作码，最多可有256条指令。只提供了183条指令，有73种编码为冗余信息。

操作码设计

扩展操作码

基本思想：指令的操作码部分采用**可变长度**的编码。
操作码的编码长度分成**几种固定长**的格式，操作码的位数**随地址数的减少而增加**，被大多数指令集采用。



PDP-11中典型指令格式

优点

- 缩短指令长度
- 减少程序总位数
- 增加指令字所能表示的操作信息

一个重要原则：

使用频度高的指令：短的操作码
使用频度低的指令：较长的操作码

扩展操作码技术

操作码的位数随地址数的减少而增加

	OP	A ₁	A ₂	A ₃	
4 位操作码	0000 0001 ⋮ 1110	A ₁ A ₁ ⋮ A ₁	A ₂ A ₂ ⋮ A ₂	A ₃ A ₃ ⋮ A ₃	15条三地址指令
8 位操作码	1111 1111 ⋮ 1111	0000 0001 ⋮ 1110	A ₂ A ₂ ⋮ A ₂	A ₃ A ₃ ⋮ A ₃	15条二地址指令
12 位操作码	1111 1111 ⋮ 1111	1111 1111 ⋮ 1111	0000 0001 ⋮ 1110	A ₃ A ₃ ⋮ A ₃	15条一地址指令
16 位操作码	1111 1111 ⋮ 1111	1111 1111 ⋮ 1111	1111 1111 ⋮ 1111	0000 0001 ⋮ 1111	16条零地址指令



设某台计算机有100条指令，

(1) 采用**固定长度操作码编码**，试设计其操作码的编码。

(2) 假如这100条指令中有**10条指令的使用概率达到90%**，其余90条指令的使用概率为10%。
试采用**不等长编码设计操作码**。



解：

(1) **固定长度**操作码编码：需**7位**操作码。

100个代码作为100条指令指令操作码即

0000000

指令0的操作码

0000001

指令1的操作码

...

1100011

指令99的操作码

剩下28个代码可用于增加新指令长度都是7位。

1100100

...

1111111



解：

(2) 可变长度操作码编码

10条指令：需4位操作码编码
0000 指令0的操作码

...

1001 指令9的操作码

90条指令：需8位操作码编码
10100000 (160)

...

11111001 (249)

地址码结构

一条指令包含1个**操作码**和多个**地址码**

零地址指令

OP

- (1) 无需操作数。如：空操作 / 停机等
- (2) 所需操作数为默认的。如：堆栈等

一地址指令

OP

A1

其地址既是源操作数地址，也是存放结果地址

- (1) 单目运算：如：取反 / 取负等
- (2) 双目运算：另一操作数为默认的 如：累加器等

地址码结构

SS 存储器-存储器类型
RS 寄存器-存储器类型
RR 寄存器-寄存器类型

一条指令包含1个**操作码**和多个**地址码**

二地址指令(最常用)



分别存放双目运算中两个源操作数地址，并将其中一个地址作为存放结果地址

三地址指令(RISC风格)



分别为双目运算中两个源操作数地址和一个结果地址

多地址指令

用于成批数据处理的指令，如：向量指令等

4.2.3 指令字长度

1. **指令字长度**: 一个指令字包含二进制代码的位数
2. **机器字长**: 计算机能直接处理的二进制数据的位数
3. **单字长指令**: 指令字长度等于机器字长度的指令
4. **半字长指令**: 指令字长度等于半个机器字长度的指令
5. **双字长指令**: 指令字长度等于两个机器字长度的指令

Pentium机的指令字长度可变，最短1个字节，最长12个字节

指令格式举例(MIPS/ARM)

1、MIPS指令格式（R型、I型、J型）

R型

6位	5位	5位	5位	5位	6位
OP	rs	rt	rd	shamt	funct

OP字段——操作码，指定一条指令的基本操作

rs字段——指定第1个源操作数寄存器，最多有32个寄存器

rt字段——指定第2个源操作数寄存器，最多有32个寄存器

rd字段——指定存放操作结果的目的地数寄存器

shamt字段——移位值，用于移位指令

funct字段——函数码，指定R型指令的特定操作

指令格式举例(MIPS/ARM)

1、MIPS指令格式（R型、I型、J型）

R型

6位	5位	5位	5位	5位	6位
OP	rs	rt	rd	shamt	funct

I型

6位	5位	5位	16位
OP	rs	rt	常数或地址(address)

J型

6位	26位
OP	地址(address)

指令格式举例(MIPS/ARM)

2、ARM的指令格式

4位	2位	1位	4位	1位	4位	4位	12位
cond	F	I	opcode	S	Rn	Rd	operand 2
		opcode					offset 12

cond字段——指明条件，该字段涉及条件转移指令

F字段——说明指令的类型

I字段——指明立即数,第2个源操作数的位置

opcode字段——指明指令的基本操作

S字段——设置状态,涉及条件转移指令

Rn字段——指明源寄存器地址(4位),共16个寄存器

Rd字段——指明目标寄存器地址(4位),共16个寄存器

operand 2字段——指明第2个源操作数

例1：P126

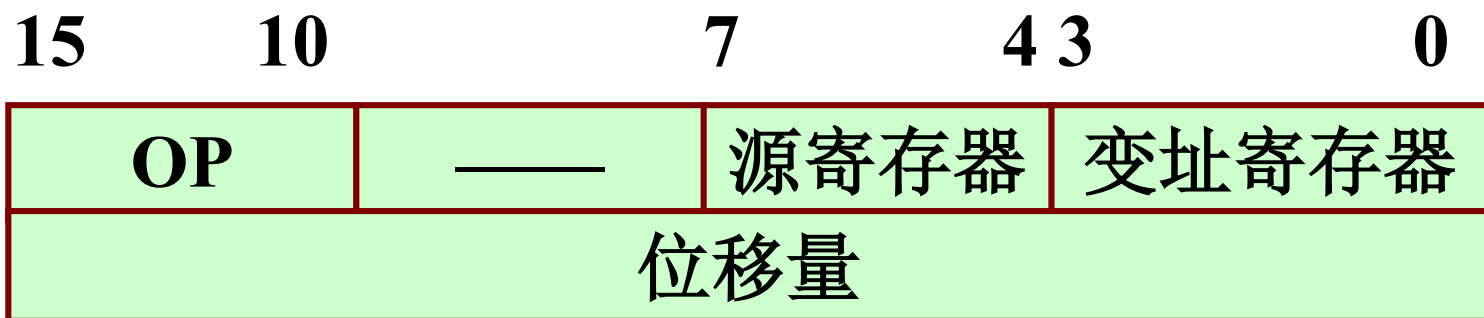
- 某机器是16位机，指令格式如下所示，OP为操作码，分析指令格式特点。



- 1) 单字长，二地址
- 2) OP可以指定？个指令 $2^7=128$
- 3) 寄存器数目？ $2^4=16$
- 4) 指令类型？RR
- 5) 常用于算术逻辑运算

例2：P126

某机器是16位机，指令格式如下所示，**OP**为操作码，分析指令格式特点。

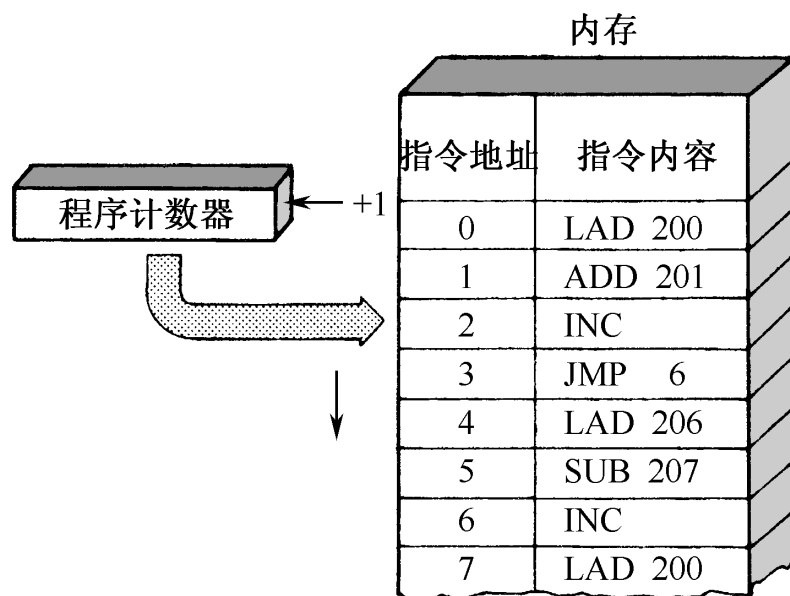


- 1) 双字长，二地址：寄存器和存储器
- 2) **OP**可以指定？个指令 $2^6=64$
- 3) 寄存器数目？ $2^4=16$
- 4) 存储器大小？ $2^{16} \times 16$ 位
- 5) 指令类型？ **RS**

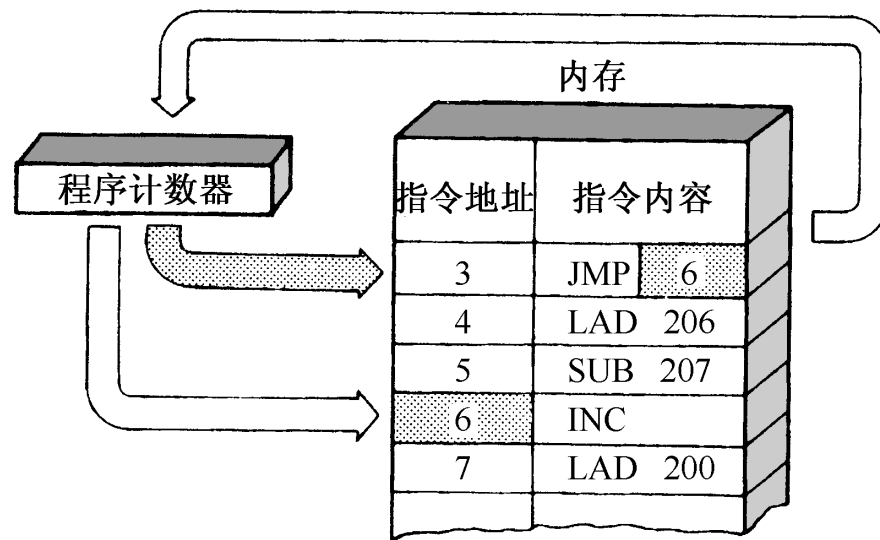
4.3 指令和数据的寻址方式

- 指令的寻址方式
- 操作数寻址方式
- 寻址方式举例

4.3.1 指令的寻址方式



(a) 指令的顺序寻址方式



(b) 指令的跳跃寻址方式

4.3.2 操作数寻址方式

- 操作数的寻址方式：形成操作数的有效地址的方法
- 指令中操作数字段的地址码是由形式地址和寻址方式特征位等组合形成
- 寻址过程就是把操作数的形式地址，变换为操作数的有效地址的过程。

操作码OP

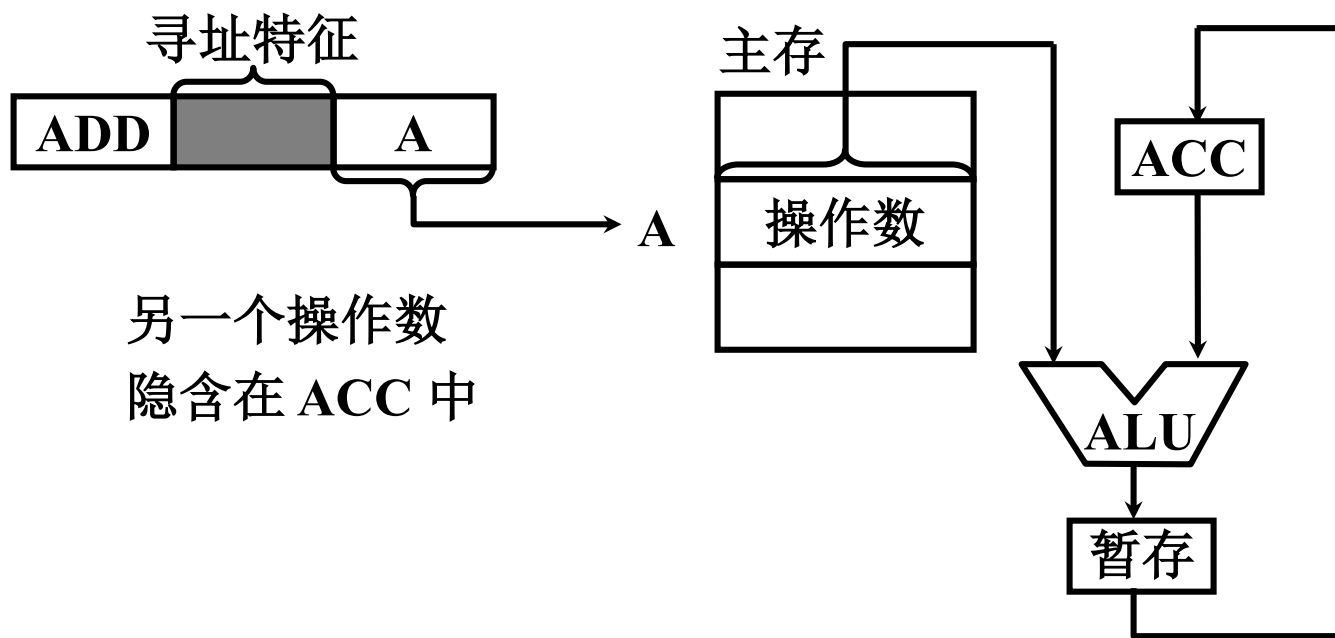
变址X

间址I

形式地址D / A

1. 隐含寻址

- 在指令中不明显的给出而是隐含着操作数的地址。

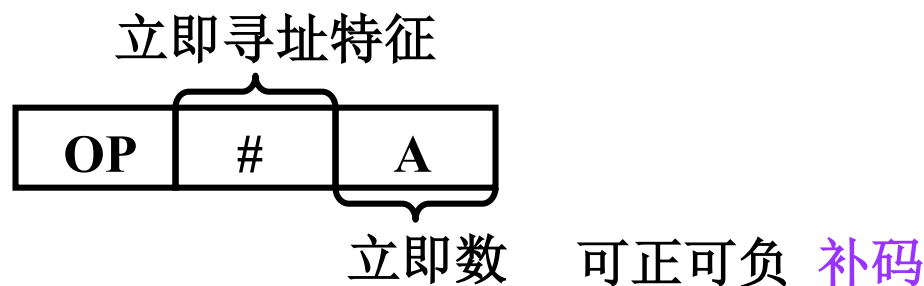


- 指令字中少了一个地址字段，可缩短指令字长

2.立即寻址

- 指令的地址字段指出的不是操作数的地址，而是操作数本身。

例：MOV AX 300H



- 指令执行阶段不访存
- A 的位数限制了立即数的范围
- 广泛使用

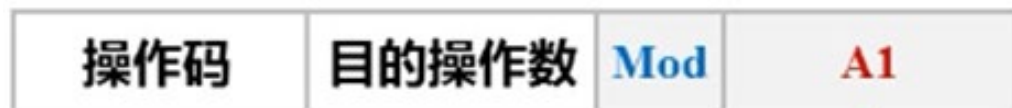
3.直接寻址

- 有效地址由形式地址直接给出
- $EA = A$

例： **MOV AX [300H]**

- 执行阶段访问一次存储器
- A 的位数决定了该指令操作数的寻址范围
- 操作数的地址不易修改（必须修改A）
- 较少使用。8位计算机和一些16位计算机。

4.间接寻址



存储器



● $EA = (A)$

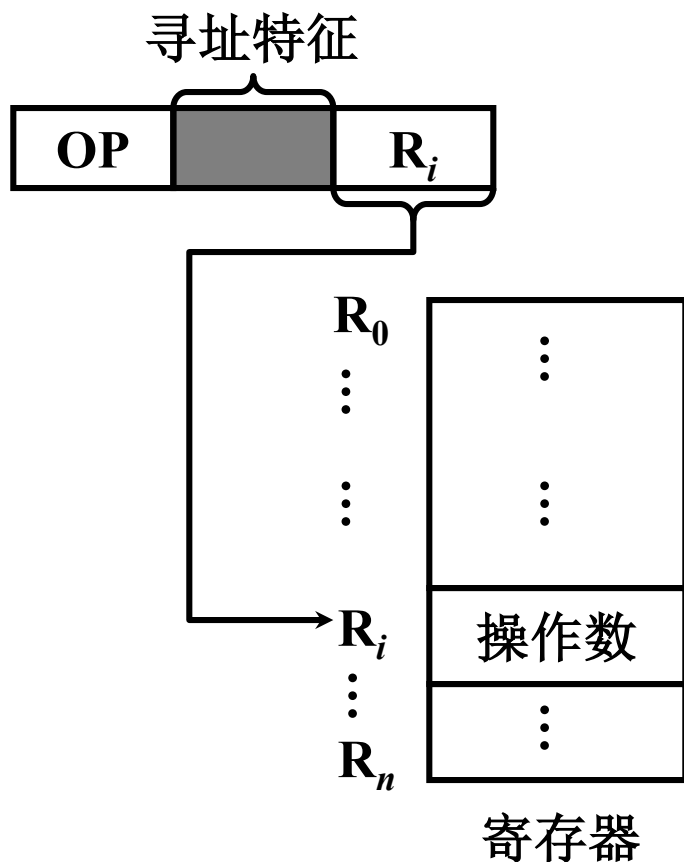
- 执行指令阶段 2 次访存
- 可扩大寻址范围
- 便于编制程序
- 执行速度慢

- 操作数和操作数地址都在存储器中
- 指令地址字段直接给出操作数地址在存储器中的地址

5.1 寄存器寻址

$$EA = R_i$$

有效地址即为寄存器编号

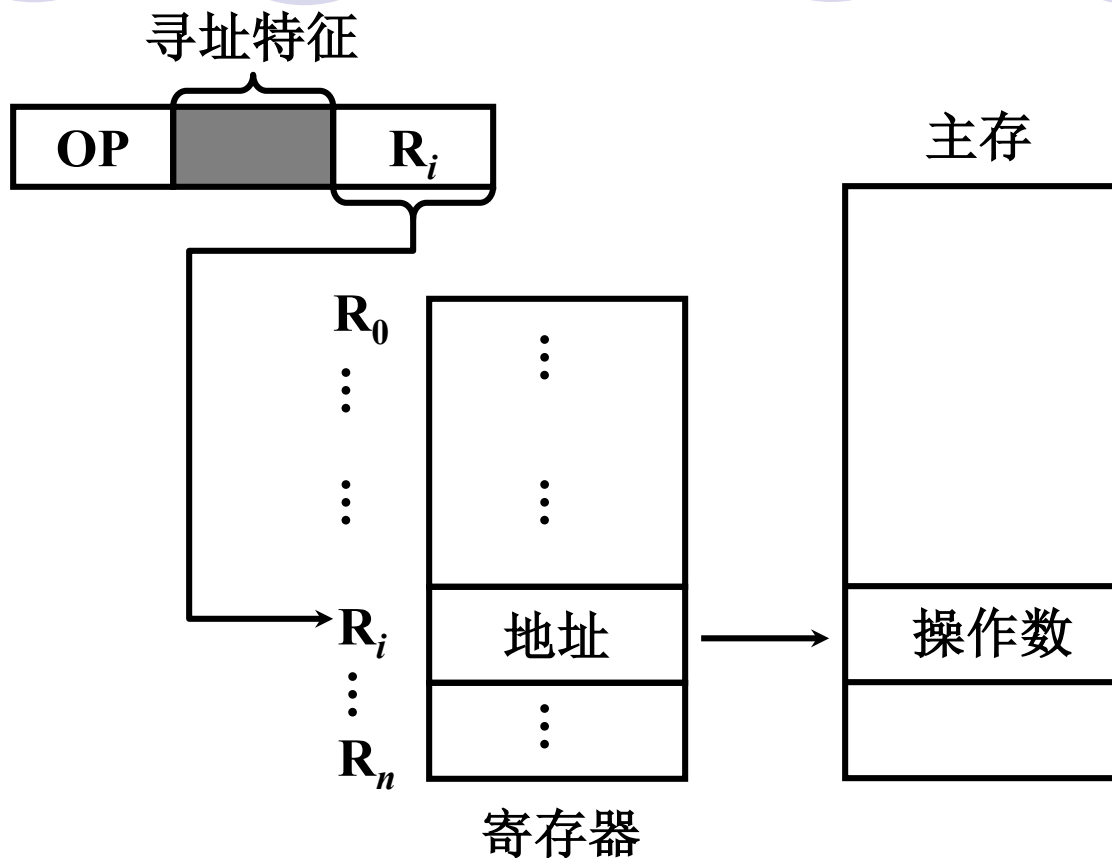


- 执行阶段不访存，只访问寄存器，执行速度快
- 寄存器个数有限，可缩短指令字长
- 使用最多，是提高性能的常用手段

5.2 寄存器间接寻址

$$EA = (R_i)$$

有效地址在寄存器中

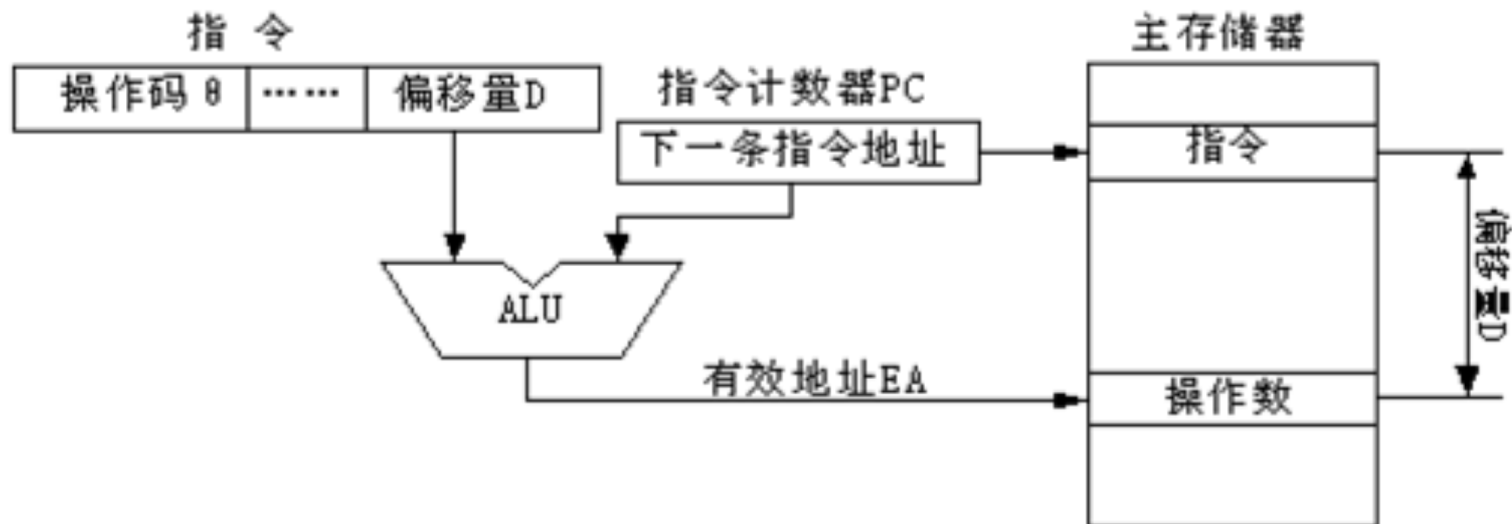


- 有效地址在寄存器中，操作数在存储器中，执行阶段访存
- 便于编制循环程序，寻址空间大，使用普遍

6. 相对寻址方式

$$EA = (PC) + A$$

A 是相对于当前指令的位移量（可正可负，补码）



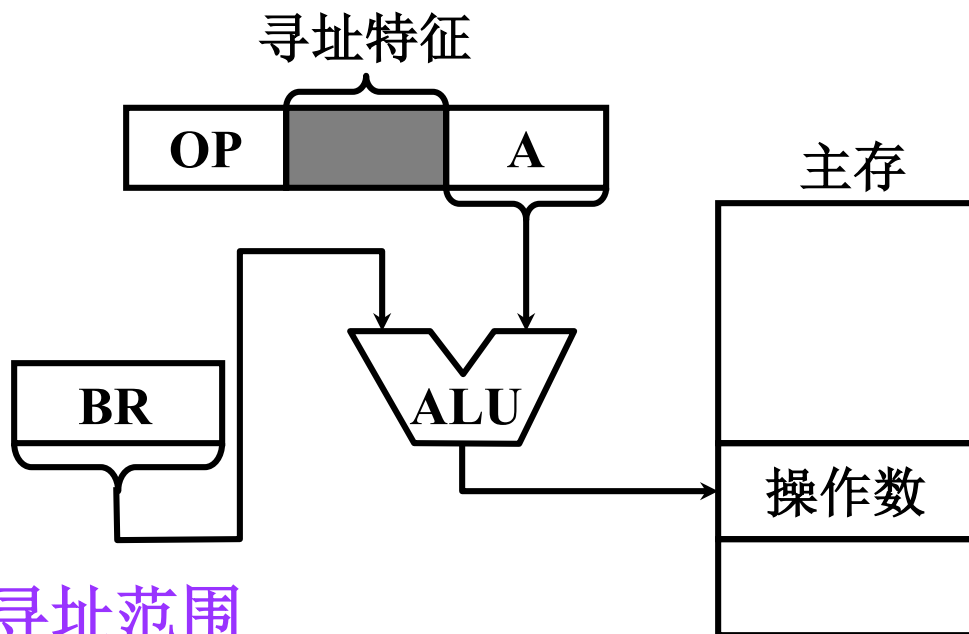
- A 的位数决定操作数的寻址范围
- 程序浮动
- 广泛用于转移指令

7. 基址寻址方式

(1) 采用专用寄存器作基址寄存器

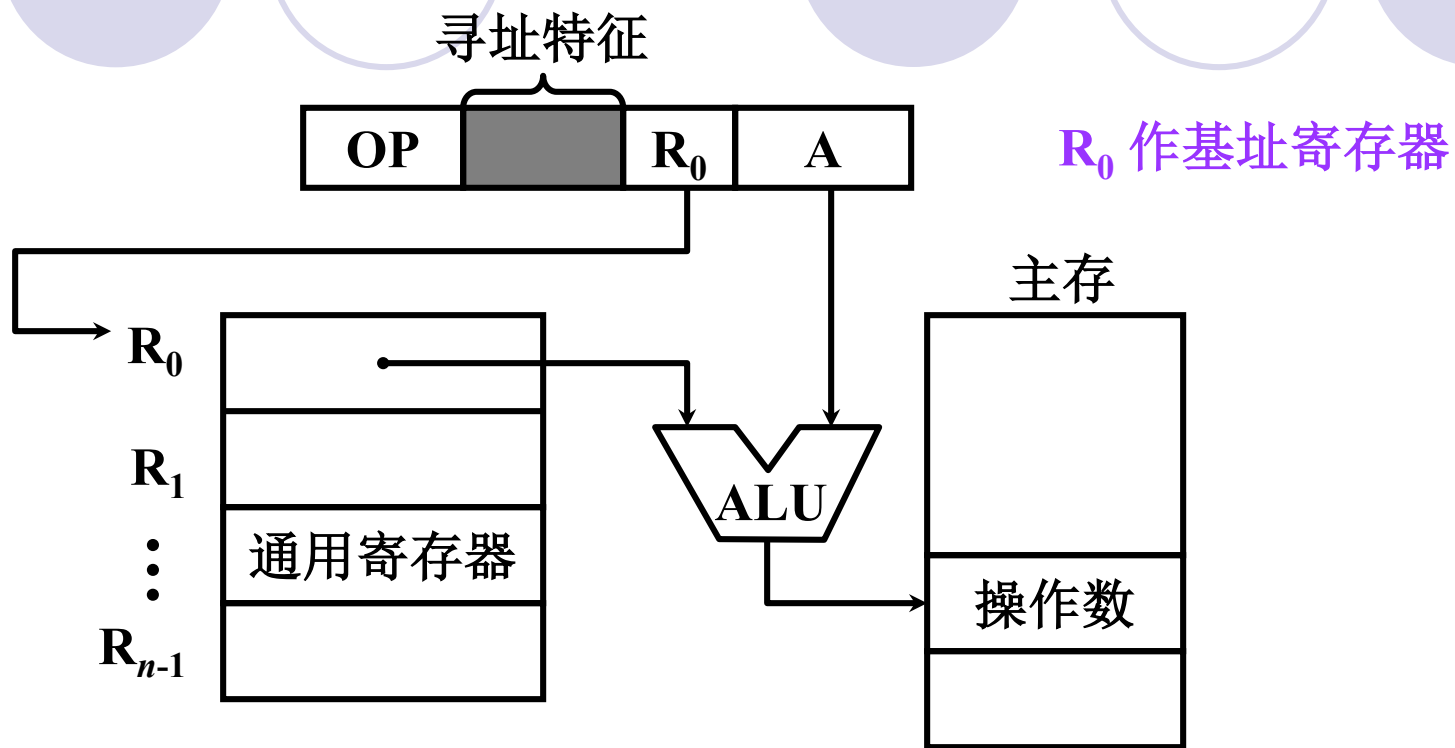
$$EA = (BR) + A$$

BR 为基址寄存器



- 可扩大寻址范围
- 便于程序搬家
- BR 内容由操作系统或管理程序确定
- 在程序的执行过程中 **BR** 内容不变，形式地址 **A** 可变

(2) 采用通用寄存器作基址寄存器



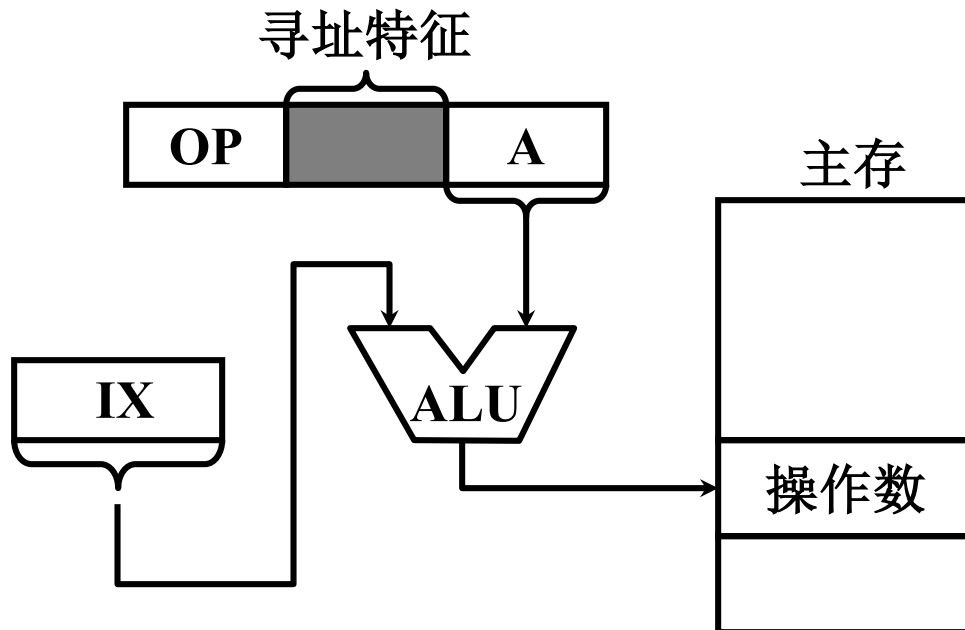
- 由用户指定哪个通用寄存器作为基址寄存器
- 基址寄存器的内容由操作系统确定
- 在程序的执行过程中 R_0 内容不变，形式地址 A 可变

8. 变址寻址方式

$$EA = (IX) + A$$

IX 为变址寄存器（专用）

通用寄存器也可以作为变址寄存器



- 可扩大寻址范围
- IX 的内容由用户给定
- 在程序的执行过程中 IX 内容可变，形式地址 A 不变
- 便于处理数组问题

寻址方式

15 下列寻址方式中，最适合按下标顺序访问一维数组元素的是（ ）。

[2017年408统考]

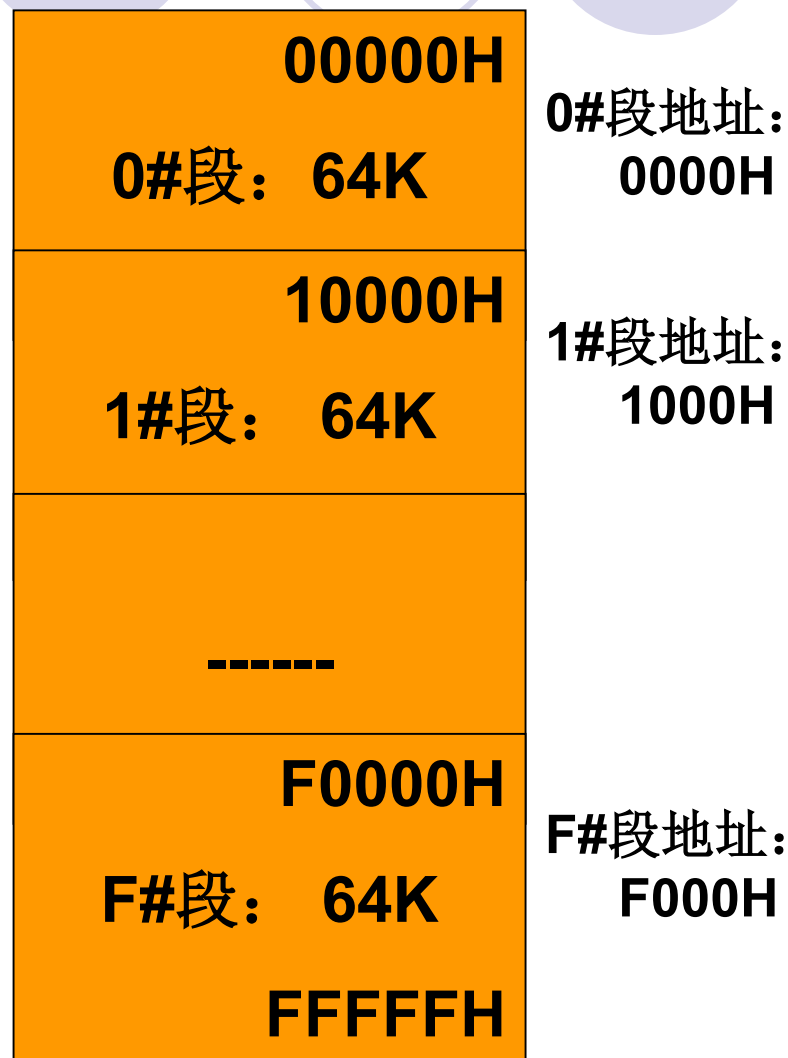
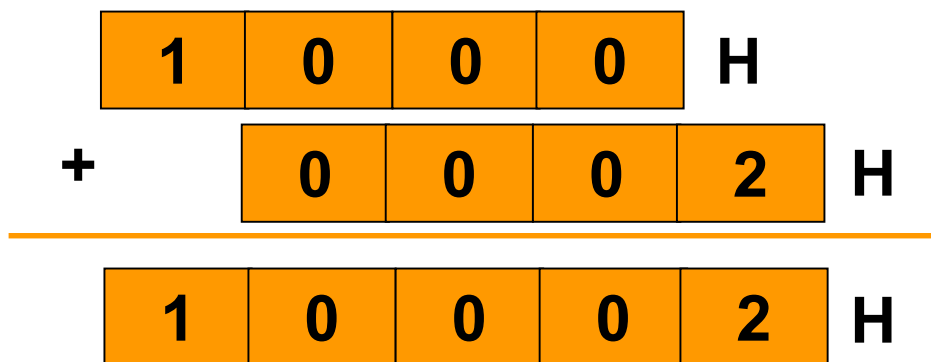
- A. 相对寻址
- B. 寄存器寻址
- C. 直接寻址
- D. 变址寻址

[答案]D

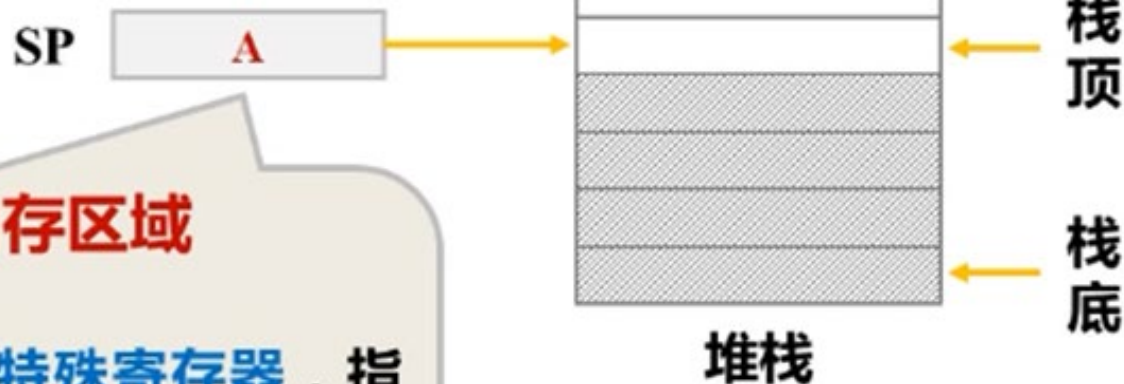
9.段寻址方式

现代计算机的主存空间普遍采用分段技术。如8086，主存地址空间1M，可划分为16个64K的段，20位物理地址由16位段地址、16位偏移地址组成。

指令地址码字段只给偏移地址。



10. 堆栈寻址方式



- 堆栈的结构：**一段内存区域**
- 栈底、栈顶
- 堆栈指针(SP)：一个**特殊寄存器**，指向栈顶

PUSH (从寄存器到堆栈)

POP (从堆栈到寄存器)

基本寻址方式

寻址方式	有效地址	操作数位置	执行阶段访存次数
立即寻址		在指令中	0
直接寻址	$EA = A$	在内存中	1
间接寻址	$EA = (A)$	在内存中	2
寄存器寻址	$EA = Ri$	在寄存器中	0
寄存器间接寻址	$EA = (Ri)$	在内存中	1
偏移寻址	$EA = (R) + A$	在内存中	1
堆栈寻址	$EA = (SP)$	在堆栈中	1

例:

寻址方式	I	X	有效地址E算法	说明
(1)	0	00	$E=D$	
(2)	0	01	$E=(PC)+/-D$	PC为程序计数器
(3)	0	10	$E=(R2)+/-D$	R2为变址寄存器
(4)	1	11	$E=(R3)$	
(5)	1	00	$E=(D)$	
(6)	0	11	$E=(R1)+/-D$	R1为基址寄存器

● 请写出六种寻址方式名称

1.直接寻址

2.相对寻址

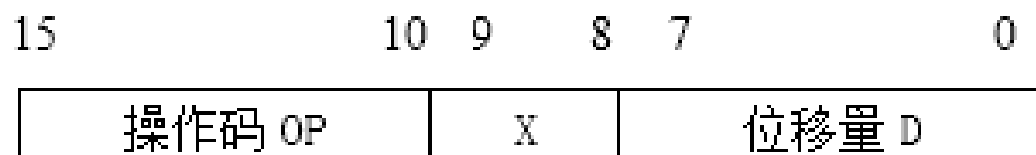
3.变址寻址

4.寄存器间接

5.间接寻址

6.基址寻址

某机的指令格式如下所示：



X 为寻址特征位：

X=00：直接寻址；

X=01：用变址寄存器 Rx1 寻址；

X=10：用变址寄存器 Rx2 寻址；

X=11：相对寻址。

设 (PC) = 1234H, (Rx1) = 0037H, (Rx2) = 1122H (H 代表十六进制数) 请确定下列指令中的有效地址：

- (1) 4422H **直接寻址, 0022H**
- (2) 2246H **变址寄存器 Rx2 寻址, 1122H+0046H=1168H**
- (3) 1324H **相对寻址, 1234H+0024H=1258H**
- (4) 3523H **变址寄存器 Rx1 寻址, 0037H+0023H=005AH**

[习题] 某16位机器所使用的指令格式和寻址方式如下所示，该机有两个20位基址寄存器，四个16位变址寄存器，十六个16位通用寄存器，指令汇编格式中的S（源）、D（目标）是通用寄存器。



要求： (1)分析三种指令的指令格式与寻址方式特点。
 (2)CPU完成哪一种操作所花时间最短？哪一种操作所花时间最长？
 第二种指令的执行时间有时会等于第三种指令的执行时间吗？

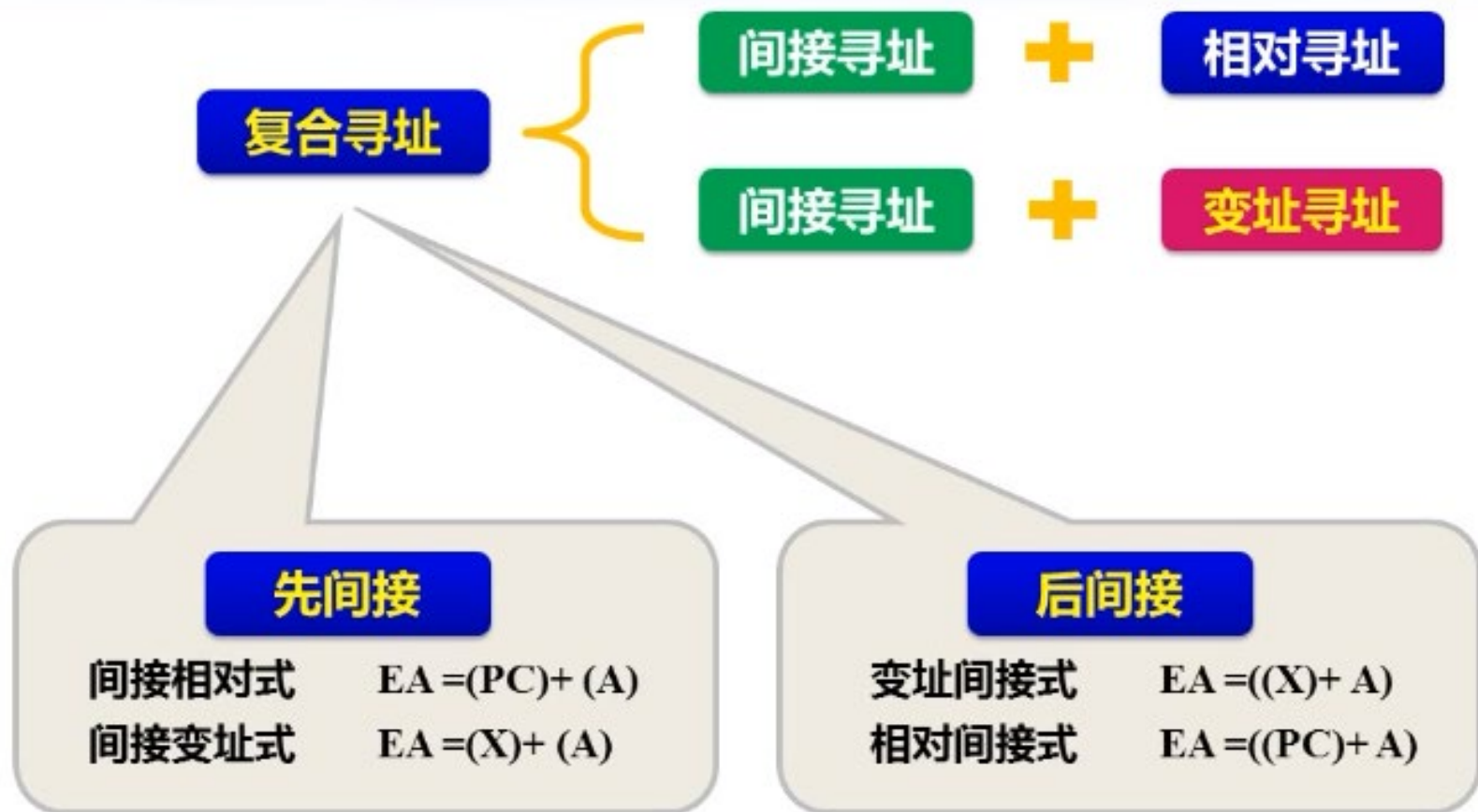
[解] 第一种指令：单字长二地址，RR型，最快；

第二种指令：双字长二地址，RS型，其中S采用基址寻址或变址寻址，需地址变换；R由源寄存器决定；且结果还需要保存到存储器；最慢；

第三种指令：双字长二地址，RS型，其中R由目标寄存器决定；S由20位直接地址决定，不需要地址变换，且是源操作数。



复合寻址方式



复合寻址方式

复合寻址

间接寻址

+

相对寻址

间接寻址

+

变址寻址

在寻址的灵活性和硬件的
复杂性之间权衡

先间接

间接相对式

$$EA = (PC) + (A)$$

间接变址式

$$EA = (X) + (A)$$

后间接

变址间接式

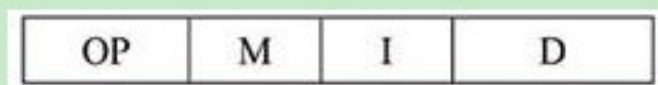
$$EA = ((X) + A)$$

相对间接式

$$EA = ((PC) + A)$$

寻址方式

36 某指令格式如下所示。



其中M为寻址方式，I为变址寄存器编号，D为形式地址。若采用先变址后间址的寻址方式，则操作数的有效地址是（ ）。[2016年408统考]

- A. $I+D$
- B. $(I)+D$
- C. $((I)+D)$
- D. $((I))+D$

[答案]C

寻址方式实例

Pentium处理器寻址方式

寻址方式

立即数(地址码A本身为操作数)

寄存器(通用寄存器的内容为操作数)

偏移量(地址码A给出8/16/32位偏移量)

基址(地址码B给出基址寄存器编号)

基址带偏移量(一维表访问)

比例变址带偏移量(一维表访问)

基址带变址和偏移量(二维表访问)

基址带比例变址和偏移量(二维表访问)

相对(给出下一指令的地址, 转移控制)

计算方法

操作数=A

操作数=(R)

$LA=(SR)+A$

$LA=(SR)+(B)$

$LA=(SR)+(B)+A$

$LA=(SR)+(I) \times S + A$

$LA=(SR)+(B)+(I) + A$

$LA=(SR)+(B)+(I) \times S + A$

转移地址=(PC)+A

LA = 线性地址; (X) = X中的内容; SR = 段寄存器;
PC = 程序计数器; B = 基址寄存器; I = 变址寄存器

4.4 RISC指令系统

- 选取使用频率最高的一些简单指令，指令条数少；
- 指令长度固定，指令格式种类少，寻址方式种类少；
- 只有取数 / 存数指令访问存储器，其余指令的操作都在寄存器之间进行。

● RISC与CISC的主要特征对比

比较内容	CISC	RISC
指令系统	复杂、庞大	简单、精简
指令数目	一般大于200	一般小于100
指令格式	一般大于4	一般小于4
寻址方式	一般大于4	一般小于4
指令字长	不固定	等长
可访存指令	不加限制	只有取数、存数指令
各种指令使用频率	相差很大	相差不大
各种指令执行时间	相差很大	绝大多数在一个周期内完成
优化编译实现	很难	较容易
程序源代码长度	较短	较长
控制器实现方式	绝大多数为微程序控制	绝大多数为硬布线控制
软件系统开发时间	较短	较长

[综合题]

- 某计算机字长为**16位**，主存地址空间大小为**128KB**，按字编址，采用定长指令格式，指令各字段定义如下：

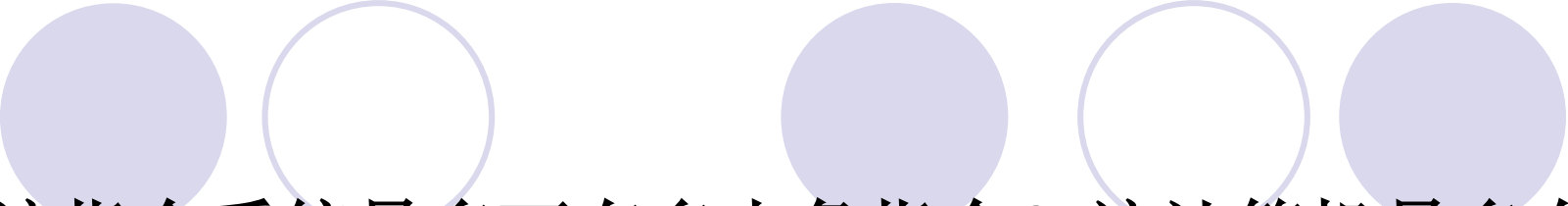


- 转移指令采用相对寻址方式，相对偏移是用补码表示，寻址方式定义如下：

Ms/Md	寻址方式	助记符	含义
000B	寄存器直接	Rn	操作数 = (Rn)
001B	寄存器间接	(Rn)	操作数 = ((Rn))
010B	寄存器间接、 自增	(Rn) +	操作数 = ((Rn)) , (Rn) + 1 → Rn
011B	相对	D (Rn)	转移目标地址 = (PC) + (Rn)

注：

(**X**) 表示存储器地址**X**或寄存器**X**的内容，请回答下列问题：



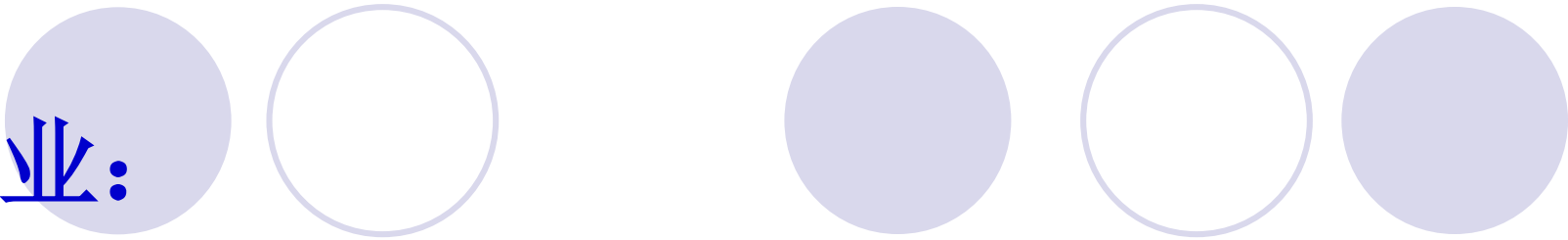
(1) 该指令系统最多可有多少条指令？该计算机最多有多少个通用寄存器？存储器地址寄存器（**MAR**）和存储器数据寄存器（**MDR**）至少各需多少位？

(2) 转移指令的目标地址范围是多少？

(3) 若操作码**0010B**表示加法操作（助记符为**add**），寄存器**R4**和**R5**的编号分别为**100B**和**101B**，**R4**的内容为**1234 H**，**R5**的内容为**5678 H**，地址**1234 H**中的内容为**5678 H**，地址**5678H**中的内容为**1234 H**，则汇编语句为**add (R4), (R5) +**（逗号前为源操作数，逗号后为目的操作数）对应的机器码是什么（用十六进制表示）？该指令执行后，哪些寄存器和存储单元的内容会改变？改变后的内容是什么？

[解答]

- (1) 该指令系统最多可有**16**条指令；该计算机最多有**8**个通用寄存器；因为主存容量**128KB**，按字编址，计算机字长为**16**位，划分为**64K**个存储单元，故**MAR**和**MDR**各需**16**位。
- (2) 由于通用寄存器为**16**位，所以转移指令的目标地址范围： $(PC) - 32768 \sim (PC) + 32767$
- (3) 汇编语句 `add (R4),(R5)+`对应的机器码为 `0010001100010101B` = `2315H`；该指令执行后，寄存器 `R5`和地址为 `5678H`的存储单元内容会改变，改变后的内容分别为： $(5678H) = 5678H + 1234H = 68ACH$ ， $(R5) = 5678H + 1 = 5679H$

The header features two groups of three circles each. The first group on the left consists of a solid light purple circle, a white circle with a light purple outline, and another solid light purple circle. The second group on the right consists of a solid light purple circle, a white circle with a light purple outline, and another solid light purple circle.

作业:

● P147-148

6、12



河海大学

Q&A